# CSE 4065 – Computational Genomics
# Assignment #2

## 1. Introduction

In this assignment we implemented dynamic pairwise sequence alignment algorithm with affine gap penalties. The algorithm finds the optimal alignment to align two DNA sequences. The optimal alignment occurs when we find the maximum score. Each matched character increases the score while mismatches and gaps are penalized.

## 2. Implementation

We firstly assigned scores and penalties as variables and then created three matrices (lower, middle, and upper), and initialized them with an arbitrary value.

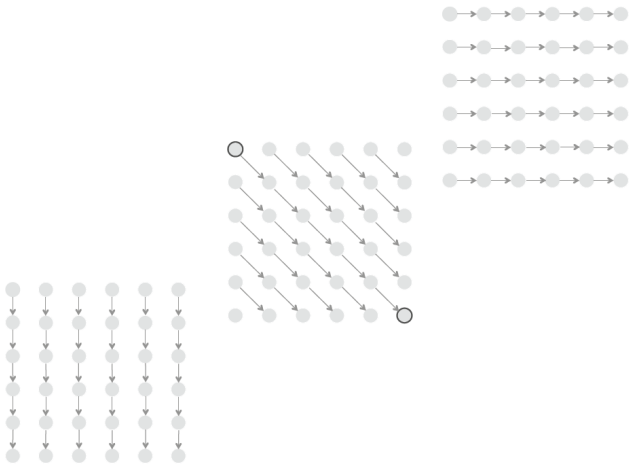


**Figure 1** Building a three-level graph for alignment with affine gap penalties. The lower level corresponds to gap extensions in the second sequence, the middle level corresponds to matches and mismatches, and the upper level corresponds to gap extensions in the first sequence. **[1]**

We initialized three matrices like in Figure 2. We first initialized (0,0) with 0 score and we found first row and column scores with adding penalties. We used negative infinity as the initial value because it is the lowest possible value. After this initialization step, we calculated scores for each node by checking previous node scores and selecting the highest score which can be seen in Figure 4.

```
int len1 = s1.Length;
int len2 = s2.Length;

// matrix to store scores for matches and missmatches
float[,] middleMatrix = new float[len1 + 1, len2 + 1];

// matrix to store scores for insertion in seq1
float[,] upperMatrix = new float[len1 + 1, len2 + 1];
for (int i = 0; i < len1 + 1; i++)
{
    for (int j = 0; j < len2 + 1; j++)
    {
        upperMatrix[i, j] = float.NegativeInfinity;
    }
}

// matrix to store scores for deletion in seq2
float[,] lowerMatrix = new float[len1 + 1, len2 + 1];
for (int i = 0; i < len1 + 1; i++)
{
    for (int j = 0; j < len2 + 1; j++)
    {
        lowerMatrix[i, j] = float.NegativeInfinity;
    }
}

middleMatrix[0, 0] = 0;
for (int i = 1; i <= len1; i++)
{
    middleMatrix[i, 0] = gap_open + ((i - 1) * gap_extend);
    upperMatrix[i, 0] = middleMatrix[i, 0];
    lowerMatrix[i, 0] = float.NegativeInfinity;
}

for (int j = 1; j <= len2; j++)
{
    middleMatrix[0, j] = gap_open + ((j - 1) * gap_extend);
    upperMatrix[0, j] = float.NegativeInfinity;
    lowerMatrix[0, j] = middleMatrix[0, j];
}
```

**Figure 2** Implementation of matrix initialization

The three-level alignment graph can be represented by the system of three recurrence relations shown in Figure 3.

$$lower_{i,j} = \max \begin{cases} lower_{i-1,j} - \epsilon \\ middle_{i-1,j} - \sigma \end{cases}$$

$$middle_{i,j} = \max \begin{cases} lower_{i,j} \\ middle_{i-1,j-1} + Score(v_i, w_j) \\ upper_{i,j} \end{cases}$$

$$upper_{i,j} = \max \begin{cases} upper_{i,j-1} - \epsilon \\ middle_{i,j-1} - \sigma \end{cases}$$

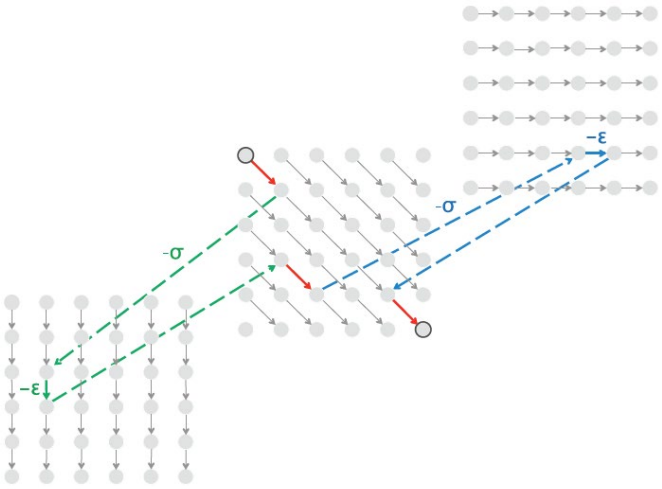



**Figure 3** Standard alignment graph **[1]**

Our implementation for the recurrence relation in Figure 3 can be seen Figure 4 below. As we defined gap penalties as negative float numbers, we do not need to do subtraction, we can sum the numbers directly. We are finding score of new node with checking nodes connected which have already scores, we are selecting node with highest possible score and adding this score gap penalties. We are looping this step until all nodes are filled with scores.

```
// fill matrices with highest possible aligment score
for (int i = 1; i <= len1; i++)
{
    // fill matrices with checking score of previous nodes
    for (int j = 1; j <= len2; j++)
    {
        upperMatrix[i, j] =
            Math.Max(middleMatrix[i - 1, j] + gap_open,
                upperMatrix[i - 1, j] + gap_extend);


        lowerMatrix[i, j] =
            Math.Max(middleMatrix[i, j - 1] + gap_open,
                lowerMatrix[i, j - 1] + gap_extend);

        middleMatrix[i, j] =
              Math.Max(middleMatrix[i - 1, j - 1] +
                  (s1[i - 1] == s2[j - 1] ? match_score : mismatch_score),
                  Math.Max(upperMatrix[i, j], lowerMatrix[i, j]));
    }
}
```

**Figure 4** Our implementation for calculating the lengths of the longest paths from the source node to $(i, j)_{upper}$, $(i,j)_{lower}$, and $(i, j)_{middle}$, respectively.

To find the optimal alignment, the code begins at the bottom right corner of the middle matrix and traces back through it. As it moves backwards, it appends the characters from the first sequence and the second sequence to the *align1* and *align2* strings to create the aligned sequences. The resulting alignment will be the optimal one. The implementation of constructing aligned sequences can be seen in Figure 5.

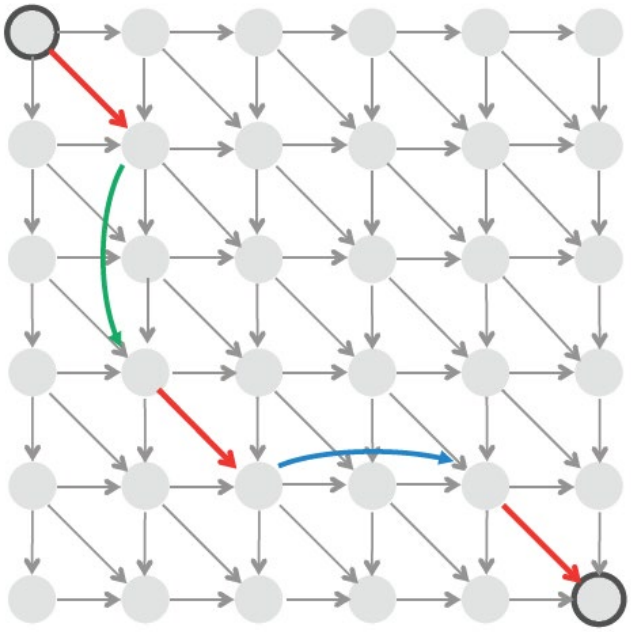


**Figure 5** Illustration of aligned sequence construction from grid. Red edges represent matches/mismatches. Blue and green edges represent insertions and deletions. **[1]**

We used *List<char>* data type to store aligned sequences because if we had used strings the time complexity of inserting a new character to the sequences would be $O(n^2)$ due to a new string will be created from index 0 after each insertion. By using list structure, we can insert new characters with O(1) time complexity. After finishing the backtracking, we can convert them to strings in O(n) time complexity.

```
// aligned sequences
List<char> align1 = new List<char>();
List<char> align2 = new List<char>();

// initialize indices to be at the bottom right corner of the matrix
int a = len1;
int b = len2;

// find the score with tracebacking from bottom right corner
// until we reach the top left corner of the matrix
while (a > 0 || b > 0)
{
    // if current node reached by matching or mismatching
    if (
        a > 0 && b > 0 && middleMatrix[a, b] == middleMatrix[a - 1, b - 1]
        + (s1[a - 1] == s2[b - 1] ? match_score : mismatch_score)
    )
    {
        //adds characters from both sequences to new alignments
        align1.Add(s1[a - 1]);
        align2.Add(s2[b - 1]);
        a -= 1;
        b -= 1;
    }
    // if current node is reached from upper matrix (insertion)
    else if (a > 0 && middleMatrix[a, b] == upperMatrix[a, b])
    {
        //adds only a character from sequence 1 and adds a gap to sequence 2
        align1.Add(s1[a - 1]);
        align2.Add('-');
        a -= 1;
    }
    // if current node is reached from lower matrix (deletion)
    else if (b > 0 && middleMatrix[a, b] == lowerMatrix[a, b])
    {
        //adds only a character from sequence 2 and adds a gap to sequence 1
        align1.Add('-');
        align2.Add(s2[b - 1]);
        b -= 1;
    }
}
```

**Figure 6** Implementation of aligned sequences construction from score matrices

## 3. Results

test1.seq

```
CTTCACAAGACGG-CT-T-TGG--TTGC-CGG-G-GGC--C-TG--G-G-CGTCATCTGT----TTC--CGGAATCA-TTCCG--GTCAGGAAAACCGGCCA----TTCAATGCACTAGTC-CCTTGGACC-AACC
C-CGGTTTGGAGAAG-C-AGCCAGAG-C-

GATGAGAAAAAGGGCTCTCTGGGATT-CACGGTGCGGCATCGTGCTGAGACG-C-TATGGGGCCTTGAACGTAAGCAATTG-GATGT-AGGATGTGTGGG-AGGGGTTC----C--T--TCGCCGTAGACATAACG
ATCG-TT---A-AATTCGAGCCAG-GTCC

189
```

Score: 189

**test2.seq**

```
TCG--AGAGTGTACGACTG-GCTGTG-G-A-GGCCGTA-GTGTGACGTAGCTT-CA-TAGCGGAACTATCCCATGGGATTTGGCC-ACGGGGCT--TATAGCGAAA-CC-CGCGTTATCCAGG--GGTCG--TAGA
TGGGGTTCT-CA-GTA-TAGTATA-AACTCTCGATGGC-GTAAGCTTCGACTGAATTGTTTCGACCTAGTTGAGCAAGCCAACAGTGTACATGTGCCTCACTACGTTCGTTGG-TGTG

TAGGTAAAGT-TA-GACAGCGCAG-GCGCACGGCCGGACGGGTCA---A--TTACAGT-G-GGTA--A-CC-ATAGG----GGCCTACCTATCTGCTA-A-CAAAATTCTCGCGCAAAC-ATGCTGG-CGCCTATA
C----T-CTGCAAGCACT--T-TCGAACTCGCGAAACCAGTAAG-TT-GA--GAC--G---CGAACTAAT--A-CAA-C-ACCAGTCT-CAGGT-CCTCACGAAGTGC-TTGGATG-G

342
```

Score: 342

**test3.seq**

```
GGTC-GCTGC-AACCCGGCCACGGAGGAGTCCCGCACGGTAAGACACCGGTCCCTTGGACGGG-GG-GAGGGGACCAG-ACATTCGG-TA-GTGAGTCCACAGCC-A-GG-GTC--TA-A---CAG-CTCC--GAA
AAGACTCGTTCACCTTTAATGTGTGCCG-AGG-AACCCAGTGCACTGTAACGAGTGGGAATCAT-G-GATGTT--GGTTACTTTCCT--CGAGGGGT--AG--CT-TTCCAGCCTG-TC--CA-CGTCCGCTTGTT
CA-CTATGAGC-GC--GCCGACCTACCCTGG---T-T---TG-C---G-A--GGTAGCGGTACTC-TATCATCATAAGCGTTTTTGCGTTCTTGGGCAATAGAAGGCCCTGATAA-TTGGCT-ACTGATTGCTAGA
C---AAA-GAGTTAGTCTG-CATCAATA--ATGTT-CATCCCCCG---G-TAA-CTGGATC-ACTGCTAATAAG-C---CTT-GAGGATACTGTTCG-CAATATTTGCTAAGT-A--TGAT---TG-T-A--GAGT
AATG---CTGGACCAT---ATGG-GGTG-TTGATCACT--TGCTAATGCACTGCAC-G-TCTTGCGACTG-ATGGT-ATGT-TTACAAAGAGGGCCGTTTGCTAAACATC-TATGA---TAGA-AGT-GACCTCCG
GTCAACC-AAGTCAC-GA-----CCACCTCAGTA-A--AAGCA-TTC-AAACT-C-GCCCTCC-AGGCTCCGAGCA-ACGCGTTTTTAGTC--GTCGCGAAGGTCACTC-CG-TAACAGGCTACCA-GCTACGTA-
A--GGC-AGAAAGGG-GC

GGTCCGCTGCGAAACCG-CCATGCAGCA--CCCACCCGCAAAGGC-CAGCTC--TTG-ACGTGTGGCG-GGGGACC-GTACACT-GCATAAGTGAGGCCT-AGCCCAAGGAG-CAATACAGTACAGACTACTAGAA
AAG-CG-G--CGCCT--ACCGTG-GACGGAGGCAAAG-AGCGCACTTTA-CG--TGGTAAT--TCGCG-TGCCCAGGT-A-TG-CAGAAC-ACGGGTCTAATACCCTTC-AGC-TGGTCTGCAAC--CCG-TTGTG
GGGCT-CGGGCAGAAAGCCG---T-CGGTGGGGGTGTGAATGACTTCGGATTGGGA-CGA-ACTCCT-TCCTC-----CGTTGTTCCGTGC--GGTT--TA-----CCA-GTTAAGTTC-CTCACGGGCTGC-GG-
CGGGGAACGA-T-AGTCGGGC-TCTCTAGTATGTGGCA-CT---GTATGATAAACTGGACCCACCG-T--TCAGTCGGACTTTGAGGCTAATAT-CGGCCTT-TTT-CT--GTCACATGTTACGTGATCACCGAG-
A-TGGATCTG--CCATCGCA-GGTGGTGCTTTAA-AGTAGTGTTA-TGCGGTCCAAAGAAGGT-C-ACTGGAT--TCA-GTCTTA----GA---C-GA---CTAACC-TCCT-TGGCCCT-GATAGTCG-CCTCC-
TTGA-CTTAAGACACCGAGTATGCCACCTTGGGATATGAA-CAACTCCAAACTGCAGCCC-CCGAAGCTCCGAGCGTACGCCATAAT-GCCCTGTC-CGCATGT-AC-CTCGATAAAACGCTAACAAGCAACGTAT
ATAGGCCAGAA-GAGTGA

1125,5
```

Score: 1125.5

**test4.seq**

```
CAGTACTGTCTGAATATCACTAACCACAGTCGGCGATCGC---TTGCTCTAACTACTGGAGGG-T---G-TGATTGCCTCCAGGGCAAGCCGCCGCAGCGGTCCCAG-T-TAAGCGCTGCAAAAGC--CGGA

--G--CT-----AATAGAAG-AACAA-A-TCTGCCTTCGTAGGTTGGTCTAGC-A-TGGAAGCCTCTTGCTGAAT---TC--GGGCA--CCTC-GC--CGGTCCTAGATATAAAC--TG-AA---CTTC---

163
```

Score: 163

**test5.seq**

```
AGTATACC-TGAGTGCCTGGATACGCCGTCTAAGCACTACGGTCT---TTCATCGTGCACCA---ACTCCATGCTTAGACGGCGCTTTCTG-AGGACGTGAAATAGAGCGCTGAGATACG-ACA-TCCCAATTGAT
G--GGC

AGTATACCGTGAGTCCCTG-ATACGAGGTC----C--T-CG--CTGAGTTC--CGTT-ACTACGTACTCCAAACTTAT--GGC--TTACTGTAGAA-GTGTA-TACAGCGCTGAGATACGGACAATCCGA-TTGAT
GAAGGA

249
```

Score: 249

## 4. References


**[1]** Compeau, P., & Pevzner, P. (2018). Bioinformatics algorithms: An active learning approach. Active Learning Publishers.